COMPREHENSIVE_TECHNICAL_SYNTHESIS

COMPREHENSIVE TECHNICAL SYNTHESIS: HelixTranslate + LLMsVerifier + HelixAgent Integration

Executive Summary

This document synthesizes deep repository analysis of three Go-based systems to provide an actionable integration blueprint. The core goal: integrate LLMsVerifier into HelixTranslate as the single source of truth for model provisioning, verification, and scoring, using HelixAgent's existing integration as the reference implementation.

Key Systems: | System | Module | Go Version | Primary Role | |--------|--------|------------|--------------| | HelixTranslate | `digital.vasic.translator` | 1.25.2 | Universal multi-format ebook translation engine | | LLMsVerifier | `digital.vasic.llmsverifier` | 1.25.3 | Enterprise LLM verification & benchmarking platform | | HelixAgent | `dev.helix.agent` | 1.26 | AI ensemble LLM service (reference integration) |

---

1. ARCHITECTURE MAPPING

1.1 Component Cross-Reference Matrix

```
HelixTranslate                    LLMsVerifier
HelixAgent
------------------------------    ------------------------------
------------------------------
cmd/api-server/main.go      <-->  api/server.go
<--> cmd/helixagent/main.go
cmd/grpc-server/main.go     <-->  (gRPC via google.golang.org)
<--> cmd/grpc-server/main.go
cmd/unified-translator/     <-->  cmd/main.go
<--> cmd/helixagent/main.go
pkg/api/handler.go          <-->  api/handlers.go
```

```
<--> internal/handlers/*.go
pkg/translator/llm/*.go     <-->  providers/*.go
<--> internal/llm/providers/*/
internal/config/config.go   <-->  config/config.go
<--> internal/config/*.go
pkg/events/events.go        <-->  api/event system
<--> internal/eventbus/*.go
pkg/verification/*.go       <-->  verification/*.go
<--> internal/verifier/*.go
pkg/websocket/hub.go        <-->  (not present)
<--> (streaming handlers)
pkg/distributed/*.go        <-->  (not present)
<--> internal/llm/orchestration
pkg/models/registry.go      <-->  providers/
model_provider_service.go <--> internal/verifier/discovery.go
```

1.2 Data Flow Architecture (Target State)

```
[HelixTranslate Application]
    |
    | 1. Config Load
    v
[internal/verifier/startup.go]  <-- NEW (modeled after HelixAgent)
    |
    | 2. Verify All Providers
    v
[digital.vasic.llmsverifier/llmverifier]
    |---> providers/ (25+ adapters)
    |---> verification/ (8-test pipeline)
    |---> scoring/ (5/7-component scoring)
    |---> database/ (SQLite + SQLCipher)
    |
    | 3. Return Verified + Scored Models
    v
[internal/services/llmsverifier_score_adapter.go]  <-- NEW
    |
    | 4. Score Normalization (0-100 -> 0-10)
    v
[pkg/translator/llm/llm.go]  <-- MODIFIED
    |
    | 5. Translation via Verified Providers
    v
[LLM Provider APIs]
```

1.3 Architectural Pattern Alignment

Pattern	HelixTranslate	LLMsVerifier	HelixAgent	Alignment Strategy
Provider Factory	`pkg/translator/`	`providers/` 25+ adapters	`internal/llm/providers/`	Unify via LLMsVerifier adapter registry

Pattern	HelixTranslate	LLMsVerifier	HelixAgent	Alignment Strategy
	`llm/llm.go` switch		`*/` per-provider	
Config Loading	`internal/config/config.go` JSON	`config/config.go` Viper (YAML/JSON/TOML)	`internal/config/` env-var centralized	Adopt Viper + env interpolation
Event Bus	`pkg/events/events.go` pub/sub	API-based events	`internal/eventbus/*.go`	Bridge LLMsVerifier events to HT event bus
Auth	JWT + API key	JWT + RBAC + LDAP	JWT + OAuth credential reading	Upgrade to LLMsVerifier RBAC
Database	PostgreSQL + Redis + SQLite	SQLite + SQLCipher	PostgreSQL + Redis	Keep PG for translation data; add SQLite for verification
Scoring	N/A (no scoring)	`scoring/` 5-component weighted	`internal/verifier/scoring.go`	Port HelixAgent scoring pattern

2. INTEGRATION POINTS INVENTORY

2.1 Files to MODIFY in HelixTranslate

#	File Path	Modification Type	Specific Changes
M1	`go.mod`	Add dependency	Add `digital.vasic.llmsverifier v0.0.0` + `replace` directive
M2	`pkg/translator/llm/llm.go`	Extend enum + factory	Add `ProviderLLMsVerifier` to Provider enum; add `ValidModels` entries; extend factory switch
M3	`pkg/translator/llm/llm.go`	Extend `LLMClient` interface	Add `GetVerificationScore() float64` method
M4	`pkg/translator/llm/llm.go`	Add import	`digital.vasic.llmsverifier/providers`
M5	`internal/config/config.go`	Add struct fields	Add `LLMsVerifierConfig` struct section
M6	`internal/config/config.go`	Add env vars	`LLMSVERIFIER_ENABLED`, `LLMSVERIFIER_API_KEY`,

#	File Path	Modification Type	Specific Changes
			`LLMSVERIFIER_BASE_URL`, `LLMSVERIFIER_DB_PATH`
M7	`pkg/api/handler.go`	Extend routes	Add `/api/v1/verify`, `/api/v1/verify/:id`, `/api/v1/translate-with-verification`
M8	`pkg/api/handler.go`	Add handler fields	Add `VerifierService`, `ScoreAdapter` fields
M9	`pkg/grpc/translator.proto`	Add RPCs	`VerifyTranslation`, `StreamVerificationProgress`
M10	`cmd/api-server/main.go`	Add initialization	Initialize verifier service, score adapter
M11	`cmd/unified-translator/main.go`	Add flags	`-verifier-enabled`, `-verifier-config`
M12	`pkg/verification/*.go`	Integration point	Wire LLMsVerifier as backend
M13	`config.json`	Add section	`"llmsverifier": { ... }` section

2.2 Files to CREATE in HelixTranslate

#	File Path	Purpose	Pattern Reference
C1	`pkg/translator/llm/llmsverifier.go`	LLMsVerifier provider adapter	`internal/llm/providers/openai/` (HelixAgent)
C2	`internal/verifier/startup.go`	Startup verification orchestrator	`internal/verifier/startup.go` (HelixAgent)
C3	`internal/verifier/config.go`	Verifier configuration types	`internal/verifier/config.go` (HelixAgent)
C4	`internal/verifier/discovery.go`	Model discovery service	`internal/verifier/discovery.go` (HelixAgent)
C5	`internal/verifier/scoring.go`	5-component scoring engine	`internal/verifier/scoring.go` (HelixAgent)
C6	`internal/verifier/enhanced_scoring.go`	7-component enhanced scoring	`internal/verifier/enhanced_scoring.go` (HelixAgent)
C7	`internal/verifier/provider_types.go`	Unified provider/model types	`internal/verifier/provider_types.go` (HelixAgent)
C8	`internal/verifier/health.go`		

#	File Path	Purpose	Pattern Reference
		Health monitoring	`internal/verifier/health.go` (HelixAgent)
C9	`internal/verifier/verification.go`	Verification service wrapper	`internal/verifier/verification.go` (HelixAgent)
C10	`internal/verifier/adapters/provider_adapter.go`	Provider interface adapter	`internal/verifier/adapters/provider_adapter.go` (HelixAgent)
C11	`internal/services/llmsverifier_score_adapter.go`	Score normalization bridge	`internal/services/llmsverifier_score_adapter.go` (HelixAgent)
C12	`pkg/verification/llmsverifier_backend.go`	Verification backend integration	`pkg/verification/` existing pattern
C13	`configs/verifier.yaml`	LLMsVerifier configuration	`configs/verifier.yaml` (HelixAgent)

2.3 Exact Function Signatures Required

```go
// C1: pkg/translator/llm/llmsverifier.go
type LLMsVerifierClient struct {
    providerService *providers.ModelProviderService
    scoringEngine *scoring.ScoringEngine
    modelID       string
    baseURL       string
    apiKey        string
}

func NewLLMsVerifierClient(config ProviderConfig)
        (*LLMsVerifierClient, error)
func (c *LLMsVerifierClient) Translate(ctx context.Context, text,
        prompt string) (string, error)
func (c *LLMsVerifierClient) GetProviderName() string
func (c *LLMsVerifierClient) GetVerificationScore() float64

// C2: internal/verifier/startup.go
type StartupVerifier struct {
    config               *StartupConfig
    verifierSvc          *VerificationService
    scoringSvc           *ScoringService
    enhancedScoring      *EnhancedScoringService
    subscriptionDetector *SubscriptionDetector
    logger               *logrus.Logger
    scoreAdapter         *ScoreAdapter
}

func NewStartupVerifier(cfg *StartupConfig, log *logrus.Logger)
        *StartupVerifier
func (sv *StartupVerifier) VerifyAllProviders(ctx
        context.Context) ([]*UnifiedProvider, error)
```

```go
func (sv *StartupVerifier) SelectDebateTeam() ([]*UnifiedModel,
        error)
func (sv *StartupVerifier) GetScoreAdapter() *ScoreAdapter

// C11: internal/services/llmsverifier_score_adapter.go
type LLMsVerifierScoreAdapter struct {
    scores     *safe.Store[map[string]float64]
    refreshMu  sync.RWMutex
    lastRefresh time.Time
}

func NewLLMsVerifierScoreAdapter() *LLMsVerifierScoreAdapter
func (a *LLMsVerifierScoreAdapter) GetProviderScore(providerID
        string) float64  // Returns 0-10
func (a *LLMsVerifierScoreAdapter) GetModelScore(modelID string)
        float64        // Returns 0-10
func (a *LLMsVerifierScoreAdapter) RefreshScores(ctx
        context.Context) error
func (a *LLMsVerifierScoreAdapter) inferProviderFromModel(modelID
        string) string
```

---

3. API SURFACE ANALYSIS

3.1 All Public APIs from LLMsVerifier That HelixTranslate Must Consume

Package: `digital.vasic.llmsverifier/llmverifier`

Function	Signature	Purpose	Call Site in HelixTranslat
`New`	`New(cfg *config.Config) *Verifier`	Create verifier	`internal/ verifier/ startup.go`
`Verify`	`(v *Verifier) Verify() ([]VerificationResult, error)`	Run full verification	`internal/ verifier/ startup.go`
`GenerateMarkdownReport`	`(v *Verifier) GenerateMarkdownReport(results []VerificationResult, dir string) error`	Generate reports	`pkg/report/`
`GenerateJSONReport`	`(v *Verifier) GenerateJSONReport(results []VerificationResult, dir string) error`	Generate reports	`pkg/report/`
`GetGlobalClient`	`(v *Verifier) GetGlobalClient() *LLMClient`	Get HTTP client	`internal/ verifier/`
`SummarizeConversation`	`(v *Verifier) SummarizeConversation(messages`	Conversation analysis	`pkg/ coordinatior`

Function	Signature	Purpose	Call Site in HelixTranslat
	[]string) (*ConversationSummary, error)		

Package: `digital.vasic.llmsverifier/providers`

Function	Signature
NewProviderRegistry	NewProviderRegistry() *ProviderRegistry
ProviderRegistry.GetConfig	(pr *ProviderRegistry) GetConfig(name string) (*ProviderConfig, bool)
ProviderRegistry.RegisterProvider	(pr *ProviderRegistry) RegisterProvider(config *ProviderConfig)
NewModelProviderService	NewModelProviderService(config string, logger *logging.Logger *ModelProviderService
ModelProviderService.RegisterProvider	(mps *ModelProviderService) RegisterProvider(providerID, baseURL, apiKey string)
ModelProviderService.GetAllModelsWithVerification	(mps *ModelProviderService) GetAllModelsWithVerification(c context.Context) (map[string] []Model, error)
ModelProviderService.GetModels	(mps *ModelProviderService) GetModels(ctx context.Context, providerID string) ([]Model, e
NewVerifiedConfigGenerator	NewVerifiedConfigGenerator(ser *EnhancedModelProviderService, logger *logging.Logger, output string) *VerifiedConfigGenerat
NewErrorClassifier	NewErrorClassifier(provider st *ErrorClassifier

Package: `digital.vasic.llmsverifier/verification`

Function	Signature
NewVerifier	NewVerifier(db *database.Datab
Verifier.Verify	(v *Verifier) Verify(ctx conte *Request) (*database.Verificat

Function	Signature
NewCodeVerificationService	NewCodeVerificationService(htt *client.HTTPClient, logger *lo *CodeVerificationService
CodeVerificationService.VerifyModelCodeVisibility	(cvs *CodeVerificationService) VerifyModelCodeVisibility(ctx modelID, providerID string, cl ProviderClientInterface) (*Cod error)
NewCodingCapabilityVerificationService	NewCodingCapabilityVerificatio *client.HTTPClient, logger *lo *CodingCapabilityVerificationS

Package: digital.vasic.llmsverifier/scoring

Function	Signature	Purp
NewScoringEngine	NewScoringEngine(db *database.Database, modelsDevClient ModelsDevClientInterface, logger interface{}) *ScoringEngine	Crea engi
ScoringEngine.CalculateComprehensiveScore	(se *ScoringEngine) CalculateComprehensiveScore(ctx context.Context, modelID string, config ScoringConfig) (*ComprehensiveScore, error)	Calc score
DefaultScoringConfig	DefaultScoringConfig() ScoringConfig	Defa weig
DefaultScoreWeights	DefaultScoreWeights() ScoreWeights	Defa weig

Package: digital.vasic.llmsverifier/config

Function	Signature	Purpose	Call Site
LoadFromFile	LoadFromFile(path string) (*Config, error)	Load config	cmd/api-server/main.go, cmd/unified-translator/main.go
SaveToFile	SaveToFile(cfg *Config, path string) error	Save config	internal/config/config.go

Package: `digital.vasic.llmsverifier/database`

Function	Signature	Purpose	Call Site
`New`	`New(path, encryptionKey string) (*Database, error)`	Open database	`cmd/api-server`
`Database.Ping`	`(db *Database) Ping() error`	Health check	`internal/healt`
`Database.CreateProvider`	`(db *Database) CreateProvider(p *Provider) (int64, error)`	CRUD	`internal/verif`
`Database.ListModels`	`(db *Database) ListModels(filters map[string]interface{}) ([]*Model, error)`	Query models	`internal/servi`
`Database.GetTopScoringModels`	`(db *Database) GetTopScoringModels(limit int) ([]*Model, error)`	Rank models	`internal/servi llmsverifier_s`

Package: `digital.vasic.llmsverifier/capabilities`

Function	Signature	Purpose	Ca
`NewDetector`	`NewDetector() *Detector`	Create detector	`ir ve di`
`Detector.DetectProviderCapabilities`	`(d *Detector) DetectProviderCapabilities(ctx context.Context, provider, apiKey string) (*ProviderCapabilities, error)`	Feature detection	`ir ve di`

3.2 REST API Endpoints (LLMsVerifier → HelixTranslate Bridge)

```
LLMsVerifier Endpoints                    HelixTranslate Bridge
----------------------                    ---------------------
GET  /api/health                    -->   GET /api/v1/verifier/
health
GET  /api/models?provider_id=       -->   GET /api/v1/verifier/
models
GET  /api/models/{id}               -->   GET /api/v1/verifier/
models/:id
POST /api/models/{id}/verify        -->   POST /api/v1/verifier/
models/:id/verify
GET  /api/providers                 -->   GET /api/v1/verifier/
providers
POST /api/providers                 -->   POST /api/v1/verifier/
providers
```

4. CONFIGURATION MAPPING

4.1 Current HelixTranslate Config (`config.json`) — Fields to Add

```jsonc
{
  "server": { /* existing */ },
  "security": { /* existing */ },
  "translation": { /* existing */ },
  "llmsverifier": {
    "enabled": true,
    "profile": "production",
    "database": {
      "path": "./helix-verifier.db",
      "encryption_key": "${VERIFIER_DB_ENCRYPTION_KEY}"
    },
    "verification": {
      "mandatory_code_check": true,
      "code_visibility_prompt": "Do you see my code?",
      "verification_timeout": "60s",
      "retry_count": 3,
      "retry_delay": "5s",
      "tests": [
        "existence",
        "responsiveness",
        "latency",
        "streaming",
        "function_calling",
        "coding_capability",
        "error_detection",
        "code_visibility"
      ]
    },
    "scoring": {
      "weights": {
        "response_speed": 0.25,
        "model_efficiency": 0.20,
        "cost_effectiveness": 0.25,
        "capability": 0.20,
        "recency": 0.10
      },
      "cache_ttl": "24h"
    },
    "providers": {
      "auto_discover": true,
      "verify_on_startup": true,
      "min_score_threshold": 5.0,
      "max_providers": 25
    }
  }
}
```

4.2 Environment Variables Map

Variable	Source System	Purpose	Destination in HelixTranslate
`OPENAI_API_KEY`	HT existing	OpenAI access	`translation.providers.openai.` + LLMsVerifier `llms[].api_key`
`ANTHROPIC_API_KEY`	HT existing	Anthropic access	Same dual-use
`DEEPSEEK_API_KEY`	HT existing	DeepSeek access	Same dual-use
`ZHIPU_API_KEY`	HT existing	Zhipu access	Same dual-use
`QWEN_API_KEY`	HT existing	Qwen access	Same dual-use
`GEMINI_API_KEY`	HT existing	Gemini access	Same dual-use
`GROQ_API_KEY`	LLMsVerifier	Groq access	NEW env var
`MISTRAL_API_KEY`	LLMsVerifier	Mistral access	NEW env var
`COHERE_API_KEY`	LLMsVerifier	Cohere access	NEW env var
`PERPLEXITY_API_KEY`	LLMsVerifier	Perplexity access	NEW env var
`TOGETHER_API_KEY`	LLMsVerifier	Together AI access	NEW env var
`XAI_API_KEY`	LLMsVerifier	xAI/Grok access	NEW env var
`CEREBRAS_API_KEY`	LLMsVerifier	Cerebras access	NEW env var
`CLOUDFLARE_API_KEY`	LLMsVerifier	Cloudflare access	NEW env var
`SILICONFLOW_API_KEY`	LLMsVerifier	SiliconFlow access	NEW env var
`LLMSVERIFIER_ENABLED`	NEW	Toggle integration	`llmsverifier.enabled`
`LLMSVERIFIER_DB_PATH`	NEW	SQLite path	`llmsverifier.database.path`
`VERIFIER_DB_ENCRYPTION_KEY`	NEW	SQLCipher key	`llmsverifier.database.encrypt`

4.3 Go Config Struct (HelixTranslate Extension)

```go
// internal/config/config.go — additions

// LLMsVerifierConfig holds the LLMsVerifier integration settings
type LLMsVerifierConfig struct {
    Enabled        bool                       `json:"enabled"`
    Profile        string                     `json:"profile"`
```

```
    Database       VerifierDatabaseConfig      `json:"database"`
    Verification   VerificationSettingsConfig
        `json:"verification"`
    Scoring        ScoringSettingsConfig       `json:"scoring"`
    Providers      VerifierProviderSettings    `json:"providers"`
}

type VerifierDatabaseConfig struct {
    Path           string `json:"path"`
    EncryptionKey  string `json:"encryption_key"`
}

type VerificationSettingsConfig struct {
    MandatoryCodeCheck     bool
        `json:"mandatory_code_check"`
    CodeVisibilityPrompt   string
        `json:"code_visibility_prompt"`
    VerificationTimeout    time.Duration
        `json:"verification_timeout"`
    RetryCount             int           `json:"retry_count"`
    RetryDelay             time.Duration `json:"retry_delay"`
    Tests                  []string      `json:"tests"`
}

type ScoringSettingsConfig struct {
    Weights   ScoreWeightsConfig `json:"weights"`
    CacheTTL  time.Duration      `json:"cache_ttl"`
}

type ScoreWeightsConfig struct {
    ResponseSpeed     float64 `json:"response_speed"`
    ModelEfficiency   float64 `json:"model_efficiency"`
    CostEffectiveness float64 `json:"cost_effectiveness"`
    Capability        float64 `json:"capability"`
    Recency           float64 `json:"recency"`
}

type VerifierProviderSettings struct {
    AutoDiscover      bool    `json:"auto_discover"`
    VerifyOnStartup   bool    `json:"verify_on_startup"`
    MinScoreThreshold float64 `json:"min_score_threshold"`
    MaxProviders      int     `json:"max_providers"`
}

// Add to existing Config struct:
type Config struct {
    // ... existing fields ...
    LLMsVerifier LLMsVerifierConfig `json:"llmsverifier"`
}
```

4.4 YAML Config File (`configs/verifier.yaml`)

```yaml
verifier:
  enabled: true
  profile: production

  database:
    path: "./helix-verifier.db"
    encryption_key: "${VERIFIER_DB_ENCRYPTION_KEY}"

  verification:
    mandatory_code_check: true
    code_visibility_prompt: "Do you see my code?"
    verification_timeout: 60s
    retry_count: 3
    retry_delay: 5s
    tests:
      - existence
      - responsiveness
      - latency
      - streaming
      - function_calling
      - coding_capability
      - error_detection
      - code_visibility

  scoring:
    weights:
      response_speed: 0.25
      model_efficiency: 0.20
      cost_effectiveness: 0.25
      capability: 0.20
      recency: 0.10
    cache_ttl: 24h

  providers:
    auto_discover: true
    verify_on_startup: true
    min_score_threshold: 5.0
    max_providers: 25
```

---

5. PROVIDER EXPANSION: 9 → 25+ Providers

5.1 Current HelixTranslate Providers (9)

#	Provider	File	Status
1	OpenAI	`pkg/translator/llm/openai.go`	Native
2	Anthropic	`pkg/translator/llm/anthropic.go`	Native

#	Provider	File	Status
3	DeepSeek	`pkg/translator/llm/deepseek.go`	OpenAI-compatible
4	Zhipu	`pkg/translator/llm/zhipu.go`	Native
5	Qwen	`pkg/translator/llm/qwen.go`	Native
6	Gemini	`pkg/translator/llm/gemini.go`	Native
7	Ollama	`pkg/translator/llm/ollama.go`	Local
8	LlamaCpp	`pkg/translator/llm/llamacpp.go`	Local
9	Mock	`pkg/translator/llm/mock.go`	Test

5.2 Additional Providers from LLMsVerifier (16+ to add)

#	Provider	LLMsVerifier File	Integration Path
10	Groq	`providers/groq.go`	Add `pkg/translator/llm/groq.go`
11	Cohere	`providers/cohere.go`	Add `pkg/translator/llm/cohere.go`
12	Mistral	`providers/mistral.go`	Add `pkg/translator/llm/mistral.go`
13	xAI (Grok)	`providers/xai.go`	Add `pkg/translator/llm/xai.go`
14	Replicate	`providers/replicate.go`	Add `pkg/translator/llm/replicate.go`
15	Cerebras	`providers/cerebras.go`	Add `pkg/translator/llm/cerebras.go`
16	Cloudflare Workers AI	`providers/cloudflare.go`	Add `pkg/translator/llm/cloudflare.go`
17	SiliconFlow	`providers/siliconflow.go`	Add `pkg/translator/llm/siliconflow.go`
18	Hyperbolic	`providers/hyperbolic.go`	Add `pkg/translator/llm/hyperbolic.go`
19	Together AI	`providers/togetherai.go`	Add `pkg/translator/llm/togetherai.go`
20	SambaNova	`providers/sambanova.go`	Add `pkg/translator/llm/sambanova.go`
21	Moonshot/Kimi	`providers/kimi.go`	Add `pkg/translator/llm/kimi.go`
22	Novita AI	`providers/novita.go`	Add `pkg/translator/llm/novita.go`
23	NLP Cloud	`providers/nlpcloud.go`	Add `pkg/translator/llm/nlpcloud.go`
24	Upstage	`providers/upstage.go`	Add `pkg/translator/llm/upstage.go`
25	Sarvam	`providers/sarvam.go`	Add `pkg/translator/llm/sarvam.go`

#	Provider	LLMsVerifier File	Integration Path
26	Modal	`providers/modal.go`	Add `pkg/translator/llm/modal.go`
27	PublicAI	`providers/publicai.go`	Add `pkg/translator/llm/publicai.go`
28	NIA	`providers/nia.go`	Add `pkg/translator/llm/nia.go`
29	Vulavula	`providers/vulavula.go`	Add `pkg/translator/llm/vulavula.go`
30	KimiCode	`providers/kimicode.go`	Add `pkg/translator/llm/kimicode.go`

5.3 Provider Factory Extension

```
// pkg/translator/llm/llm.go - Factory switch extension
func NewLLMTranslator(config ProviderConfig) (LLMTranslator,
        error) {
    // ... existing validation ...

    switch config.Provider {
    // ... existing 9 cases ...

    // NEW: LLMsVerifier-verified providers
    case ProviderGroq:
        client, err = NewGroqClient(config)
    case ProviderCohere:
        client, err = NewCohereClient(config)
    case ProviderMistral:
        client, err = NewMistralClient(config)
    case ProviderXAI:
        client, err = NewXAIClient(config)
    case ProviderReplicate:
        client, err = NewReplicateClient(config)
    case ProviderCerebras:
        client, err = NewCerebrasClient(config)
    case ProviderCloudflare:
        client, err = NewCloudflareClient(config)
    case ProviderSiliconFlow:
        client, err = NewSiliconFlowClient(config)
    case ProviderHyperbolic:
        client, err = NewHyperbolicClient(config)
    case ProviderTogetherAI:
        client, err = NewTogetherAIClient(config)
    case ProviderSambaNova:
        client, err = NewSambaNovaClient(config)
    case ProviderKimi:
        client, err = NewKimiClient(config)
    case ProviderNovita:
        client, err = NewNovitaClient(config)
    case ProviderNLPCloud:
```

```
        client, err = NewNLPCloudClient(config)
    case ProviderUpstage:
        client, err = NewUpstageClient(config)
    case ProviderSarvam:
        client, err = NewSarvamClient(config)
    case ProviderModal:
        client, err = NewModalClient(config)
    case ProviderPublicAI:
        client, err = NewPublicAIClient(config)
    case ProviderNIA:
        client, err = NewNIAClient(config)
    case ProviderVulavula:
        client, err = NewVulavulaClient(config)
    case ProviderKimiCode:
        client, err = NewKimiCodeClient(config)
    default:
        return LLMTranslator{}, fmt.Errorf("unknown provider:
        %s", config.Provider)
    }
}
```

5.4 ValidModels Map Extension

```
// pkg/translator/llm/llm.go
var ValidModels = map[Provider][]string{
    // ... existing mappings ...
    ProviderGroq:       {"llama-3.3-70b", "llama-3.1-8b",
        "mixtral-8x7b"},
    ProviderCohere:     {"command-r", "command-r-plus",
        "command"},
    ProviderMistral:    {"mistral-large", "mistral-medium",
        "mistral-small"},
    ProviderXAI:        {"grok-beta", "grok-2"},
    ProviderReplicate:  {"meta/llama-2-70b"},
    ProviderCerebras:   {"llama-3.3-70b"},
    ProviderCloudflare: {"@cf/meta/llama-2-7b"},
    ProviderSiliconFlow:{"deepseek-ai/DeepSeek-V2.5"},
    ProviderTogetherAI: {"meta-llama/Llama-3-70b", "mistralai/
        Mixtral-8x7B"},
    ProviderSambaNova:  {"llama-3.3-70b"},
    ProviderKimi:       {"moonshot-v1-8k", "moonshot-v1-32k"},
    ProviderNovita:     {"llama-3.3-70b"},
    // ... etc
}
```

---

6. CAPABILITY INTEGRATION MAP

6.1 MCPs (Model Context Protocol) Integration

Current State:

- HelixTranslate: NO MCP support
- LLMsVerifier: Capability detection for MCP support (`FeatureDetectionResult.MCPs bool`)
- HelixAgent: 35 MCP implementations via `MCP/` submodule + SSE bridge on port 8103

Integration Path:

```
1. Add MCP submodule: git submodule add MCP/
2. File: `pkg/mcp/client.go` - NEW, wraps MCP client
3. File: `internal/mcp/registry.go` - NEW, registers MCP servers
4. Integration in `pkg/translator/translator.go`:
   - Before translation: query MCP for context enrichment
   - After translation: use MCP for terminology validation
5. Config: Add `"mcp_servers"` array to `config.json`
```

6.2 LSPs (Language Server Protocol) Integration

Current State:

- HelixTranslate: NO LSP support
- LLMsVerifier: Capability detection for LSP (`FeatureDetectionResult.LSPs bool`)
- HelixAgent: 10 LSP servers (gopls, pylsp, typescript-language-server, rust-analyzer, clangd)

Integration Path:

```
1. File: `pkg/lsp/client.go` - NEW, LSP client wrapper
2. File: `internal/lsp/registry.go` - NEW, LSP server registry
3. Integration in `pkg/translator/llm/*.go`:
   - Code-aware translation: Use LSP for source code context
   - Language detection enhancement via LSP
4. Config: Add `"lsp_servers"` array to `config.json`
```

6.3 ACPs (Agent Coordination Protocol) Integration

Current State:

- HelixTranslate: NO ACP support
- LLMsVerifier: Capability detection (`FeatureDetectionResult.ACPs bool`)
- HelixAgent: Full ACP manager on port 8300, JSON-RPC protocol, tool calling + context management

Integration Path:

```
1. File: `pkg/acp/client.go` - NEW, ACP client
2. File: `internal/acp/manager.go` - NEW, ACP manager
3. Integration points:
   - `pkg/coordination/`: Multi-LLM coordination via ACP
   - `pkg/translator/`: Translation agent orchestration
4. Config: Add `"acp"` section to `config.json`
```

6.4 Embeddings Integration

Current State:

- HelixTranslate: NO embeddings support
- LLMsVerifier: Capability detection (`FeatureDetectionResult.Embeddings bool`)
- HelixAgent: 13 embedding providers via `internal/embeddings/`

Integration Path:

```
1. File: `pkg/embeddings/client.go` - NEW, embedding client
2. File: `pkg/embeddings/providers.go` - NEW, provider registry
3. Use cases:
   - RAG-based translation memory
   - Semantic similarity for quality verification
   - Terminology extraction
4. Config: Add `"embeddings"` section to `config.json`
```

6.5 RAG (Retrieval-Augmented Generation) Integration

Current State:

- HelixTranslate: NO RAG support
- LLMsVerifier: NO direct RAG support
- HelixAgent: `RAG/` submodule, ChromaDB/Qdrant/Neo4j, chunking (1000 tokens, 200 overlap, top_k=5)

Integration Path:

```
1. File: `pkg/rag/engine.go` - NEW, RAG engine
2. File: `pkg/rag/vector_store.go` - NEW, vector store interface
3. Use cases:
   - Translation memory retrieval
   - Terminology consistency enforcement
   - Style guide adherence
4. Config: Add `"rag"` section to `config.json`
```

6.6 Skills Integration

Current State:

- HelixTranslate: NO skills system
- LLMsVerifier: Capability-based skill inference from `VerificationResult`
- HelixAgent: Skill definitions in provider configs, capability detection, specialization tagging

Integration Path:

```
1. File: `pkg/skills/registry.go` - NEW, skill registry
2. File: `pkg/skills/detector.go` - NEW, skill detection from
LLMsVerifier results
3. Skill types: `translation`, `terminology`,
`cultural_adaptation`, `quality_verification`
4. Config: Add `"skills"` section to `config.json`
```

6.7 Plugins Integration

Current State:

- HelixTranslate: NO plugin system
- LLMsVerifier: NO plugin system
- HelixAgent: `Plugins/` submodule, hot reloading, auto-discovery

Integration Path:

```
1. File: `pkg/plugin/registry.go` - NEW, plugin registry
2. File: `pkg/plugin/loader.go` - NEW, plugin loader with hot
reload
3. Plugin types: pre-processors, post-processors, format
converters, quality checkers
4. Config: Add `"plugins"` section to `config.json`
```

6.8 Capability Detection Integration Flow

```
[LLMsVerifier Verification]
    |
    v
[FeatureDetectionResult]
    ├── MCPs: bool          → Enable/disable MCP features
    ├── LSPs: bool          → Enable/disable LSP features
    ├── ACPs: bool          → Enable/disable ACP features
    ├── Embeddings: bool    → Enable embedding-based features
    ├── Streaming: bool     → Enable real-time translation
    ├── JSONMode: bool      → Enable structured output
    ├── Reasoning: bool     → Enable complex translation logic
    ├── CodeGeneration: bool → Enable code-aware translation
    └── Multimodal: bool    → Enable image/video translation
    |
    v
[HelixTranslate Feature Gates]
    - Each capability gates specific features
    - Scores influence feature priority
    - Unverified capabilities disabled by default
```

---

7. CODE REFERENCES

7.1 HelixTranslate Specific References

File	Line(s)	Context	Action
`go.mod`	N/A	Module definition	Add `require digital.vasic.llmsverifier v0.0.0` + `replace`
`pkg/translator/llm/llm.go`	~30-50	`LLMClient` interface	Add `GetVerificationScore() float64`
`pkg/translator/llm/llm.go`	~55-70	`Provider` enum	Add 21 new provider constants
`pkg/translator/llm/llm.go`	~75-120	`NewLLMTranslator()` factory	Add 21 new switch cases
`pkg/translator/llm/llm.go`	~125-150	`ValidModels` map	Add entries for 21 new providers
`internal/config/config.go`	~40-80	`Config` struct	Add `LLMsVerifier` field
`internal/config/config.go`	~200-250	`LoadConfig()` function	Add verifier config loading
`pkg/api/handler.go`	~40-60	`Handler` struct	Add `verifierSvc`, `scoreAdapter` fields
`pkg/api/handler.go`	~100-150	`SetupRoutes()`	Add verifier route handlers
`pkg/verification/verifier.go`	N/A	Existing verification	Wire LLMsVerifier backend
`cmd/api-server/main.go`	~1-50	`main()` function	Add verifier initialization
`cmd/unified-translator/main.go`	~50-100	Flag definitions	Add `-verifier-*` flags
`config.json`	N/A	Root config	Add `"llmsverifier"` section

7.2 LLMsVerifier Specific References

File	Line(s)	Context	Action
`llm-verifier/llmverifier/verifier.go`	~24-30	`New()` constructor	Import and call from HelixTranslate

File	Line(s)	Context	Action
`llm-verifier/llmverifier/verifier.go`	~154-241	`Verify()` method	Core verification orchestration
`llm-verifier/llmverifier/verifier.go`	~306-393	`verifySingleModel()`	Single model verification logic
`llm-verifier/providers/base.go`	~8-62	`BaseAdapter` struct	Embed in new provider clients
`llm-verifier/providers/service.go`	N/A	`ProviderService` interface	Implement for HelixTranslate
`llm-verifier/providers/model_provider_service.go`	N/A	3-tier discovery	Use for model discovery
`llm-verifier/scoring/scoring_engine.go`	~37-132	`CalculateComprehensiveScore()`	Core scoring logic
`llm-verifier/scoring/scoring_engine.go`	~356-364	`DefaultScoreWeights()`	Import default weights
`llm-verifier/config/config.go`	N/A	`Config` struct	Map from HelixTranslate config
`llm-verifier/verification/verification.go`	N/A	Core verification	Use as verification backend
`llm-verifier/verification/code_verification.go`	N/A	Code visibility test	Mandatory integration
`llm-verifier/database/database.go`	N/A	SQLite + SQLCipher	Initialize for local state

7.3 HelixAgent Reference Implementation

File	Line(s)	Context	Reference Value
`cmd/helixagent/main.go`	~1-100	Startup flow	Exact initialization order
`cmd/helixagent/main.go`	N/A	Import	`digital.vasic.llmsv pkg/cliagents`
`internal/verifier/startup.go`	~1-50	`NewStartupVerifier()`	Constructor pattern
`internal/verifier/startup.go`	~51-200	`VerifyAllProviders()`	5-phase pipeline
`internal/verifier/startup.go`	N/A	Import	`digital.vasic.llmsv api_keys`
`internal/verifier/config.go`	N/A	Config types	Copy config struct patter
`internal/verifier/discovery.go`	N/A	Discovery service	Copy discovery patterns
`internal/verifier/scoring.go`	N/A	5-component scoring	Copy scoring implement
`internal/verifier/enhanced_scoring.go`	N/A	7-component scoring	Copy enhanced scoring

File	Line(s)	Context	Reference Value
`internal/verifier/provider_types.go`	N/A	Unified types	Copy type definitions
`internal/verifier/health.go`	N/A	Health monitoring	Copy health check patter
`internal/services/llmsverifier_score_adapter.go`	N/A	Score adapter	Exact implementation pa
`configs/verifier.yaml`	N/A	YAML config	Copy and adapt

8. RISK ASSESSMENT

8.1 Technical Risks Matrix

Risk	Severity	Probability	Impact	Mitigation
CGO dependency for SQLite	High	High	Build complexity	Add build tags; provide pure-Go fallback; document CGO requirement
Provider API key explosion	Medium	High	Config complexity	Unified key management; env var interpolation; secure vault integration
Test coverage drop	High	High	Quality regression	Parallel test writing; coverage gates; anti-puff enforcement
Breaking changes in LLMsVerifier API	Medium	Medium	Integration failure	Pin submodule commit; integration tests; version compatibility layer
Performance degradation	Medium	Medium	Slower translation	Async verification; score caching; lazy provider discovery
Memory bloat from dual databases	Medium	Low	Resource consumption	Shared connection pooling; SQLite WAL mode; cleanup routines
OAuth token restrictions	Medium	Medium	Auth failures	Mark restricted tokens; exclude from provider list; clear error messages
Provider rate limiting during verification	Low	High	Verification failures	Configurable rate limits; staggered

Risk	Severity	Probability	Impact	Mitigation
				verification; retry with backoff

8.2 Mitigation Strategies

```
RISK-001: CGO Dependency
- Strategy: Add `//go:build cgo` tags to SQLite-dependent files
- Fallback: Use `modernc.org/sqlite` (pure Go) for builds without
CGO
- File: Create `pkg/database/sqlite_cgo.go` and `pkg/database/
sqlite_nocgo.go`

RISK-002: API Key Management
- Strategy: Unified key storage in environment only
- Implementation: `internal/config/env_loader.go` with
interpolation
- Security: Keys NEVER committed; `.gitignore` enforced

RISK-003: Test Coverage
- Strategy: Coverage ratchet - no PR reduces coverage
- Implementation: `Makefile` target `make coverage-check`
- Gate: Fail CI if coverage < 43.6% (current) → target 80%+

RISK-004: API Stability
- Strategy: Submodule pin at known-good commit
- Implementation: `git submodule update --init -- LLMsVerifier` at
specific hash
- Fallback: Wrapper layer that adapts API changes

RISK-005: Performance
- Strategy: Verification runs async on startup, NOT blocking
- Implementation: Goroutine with context cancellation
- Cache: `safe.Store` for scores with configurable TTL

RISK-006: Memory
- Strategy: SQLite with WAL mode, connection pooling
- Implementation: `PRAGMA journal_mode=WAL; PRAGMA
synchronous=NORMAL`
- Monitor: Add memory usage metrics to WebSocket dashboard
```

---

9. IMPLEMENTATION SEQUENCE

Phase 1: Foundation (Week 1)

Goal: Go module integration, configuration system, basic scaffolding

#	Task	Files	Deliverable
1.1	Add LLMsVerifier as git submodule	`.gitmodules`, `go.mod`	Submodule initialized
1.2	Add go.mod require + replace	`go.mod`	Module importable
1.3	Extend Config struct	`internal/config/config.go`	Config types defined
1.4	Add env var loading	`internal/config/config.go`	Env vars loaded
1.5	Create `configs/verifier.yaml`	NEW	YAML config template
1.6	Update `config.json` schema	`config.json`	Schema documented

Anti-Puff Test: Build succeeds; `go mod tidy` clean; config loads from YAML

Phase 2: Core LLMsVerifier Integration (Week 2)

Goal: Provider discovery, verification, scoring operational

#	Task	Files	Deliverable
2.1	Create `internal/verifier/config.go`	NEW	Config types for verifier
2.2	Create `internal/verifier/provider_types.go`	NEW	UnifiedProvider, UnifiedModel types
2.3	Create `internal/verifier/startup.go`	NEW	StartupVerifier orchestrator
2.4	Create `internal/verifier/discovery.go`	NEW	Model discovery service
2.5	Create `internal/verifier/scoring.go`	NEW	5-component scoring
2.6	Create `internal/verifier/verification.go`	NEW	Verification service wrapper
2.7	Create `internal/verifier/health.go`	NEW	Health monitoring
2.8	Create `internal/services/llmsverifier_score_adapter.go`	NEW	Score normalization bridge
2.9	Initialize database on startup	`cmd/api-server/main.go`	SQLite initialized

Anti-Puff Test: Startup runs verification pipeline; scores stored in DB; adapter returns normalized scores

Phase 3: Provider Expansion (Week 3)

Goal: Add 21+ new providers via LLMsVerifier adapters

#	Task	Files	Deliverable
3.1	Extend `Provider` enum	`pkg/translator/llm/llm.go`	All 30+ providers defined
3.2	Add provider constants	`pkg/translator/llm/llm.go`	`ProviderGroq` through `ProviderKimiCode`
3.3	Extend factory switch	`pkg/translator/llm/llm.go`	All providers constructable
3.4	Extend `ValidModels`	`pkg/translator/llm/llm.go`	Model lists populated
3.5	Create provider client stubs (batch)	`pkg/translator/llm/*.go` (21 files)	Each client implements `LLMClient`
3.6	Add provider API key env vars	`internal/config/config.go`	All keys configurable

Anti-Puff Test: Each provider instantiates; at least 3 providers make live API calls successfully

Phase 4: API Integration (Week 4)

Goal: REST/gRPC endpoints for verification

#	Task	Files	Deliverable
4.1	Extend `Handler` struct	`pkg/api/handler.go`	Verifier fields added
4.2	Add verification routes	`pkg/api/handler.go`	`/api/v1/verify/*` registered
4.3	Implement route handlers	`pkg/api/handler.go`	All handlers functional
4.4	Extend gRPC proto	`pkg/grpc/translator.proto`	Verification RPCs added
4.5	Regenerate protobuf	`pkg/grpc/*.go`	gRPC code regenerated
4.6	Add CLI flags	`cmd/unified-translator/main.go`	`-verifier-*` flags added
4.7	Wire initialization	`cmd/api-server/main.go`	Full startup pipeline

Anti-Puff Test: `curl /api/v1/verifier/health` returns 200; verify endpoint triggers actual verification

Phase 5: Capability Integration (Week 5-6)

Goal: MCP, LSP, ACP, Embeddings, RAG, Skills, Plugins

#	Task	Files	Deliverable
5.1	Create MCP client	`pkg/mcp/client.go`	MCP integration
5.2	Create LSP client	`pkg/lsp/client.go`	LSP integration
5.3	Create ACP client	`pkg/acp/client.go`	ACP integration
5.4	Create embeddings client	`pkg/embeddings/client.go`	Embeddings integration
5.5	Create RAG engine	`pkg/rag/engine.go`	RAG integration
5.6	Create skills registry	`pkg/skills/registry.go`	Skills system
5.7	Create plugin system	`pkg/plugin/registry.go`	Plugin system
5.8	Wire capability gates	`pkg/translator/translator.go`	Feature gating by capability

Anti-Puff Test: Each capability has a working challenge script that validates real functionality

Phase 6: Quality & Documentation (Week 7)

Goal: Testing, challenges, documentation

#	Task	Files	Deliverable
6.1	Write unit tests for all new packages	`*_test.go` files	>80% coverage
6.2	Write integration tests	`test/integration/verifier_*_test.go`	Real infrastructure
6.3	Write challenge scripts	`challenges/scripts/helixtranslate_verifier_*.sh`	5+ scripts
6.4	Update AGENTS.md	`AGENTS.md`	Integration documented
6.5	Update CLAUDE.md	`CLAUDE.md`	Hard stops documented
6.6	Update API docs	`Documentation/API.md`	New endpoints documented
6.7	E2E workflow validation	`test/e2e/`	Full workflow passes

10. ANTI-PUFF TESTING REQUIREMENTS

10.1 Testing Philosophy (from HelixAgent Constitution)

```
CONST-002a: NO mocks/stubs in production code
CONST-002:  100% test coverage across ALL test types
```

```
CONST-030:  Real infrastructure for ALL non-unit tests
Rule:        "Mocks/stubs ONLY in unit tests; all other tests use
real data and live services"
Enforcement: `make no-mocks-above-unit` with strict allowlist
ratchet
```

10.2 Test Categories & Requirements

Category	Location	Mocks Allowed	Infrastructure	Coverage Target
Unit	`*_test.go` alongside source	Yes (interfaces)	None	80%+
Integration	`test/integration/`	No	Real SQLite, mock HTTP	70%+
E2E	`test/e2e/`	No	Full stack running	Full workflows
Security	`test/security/`	No	Real services	All endpoints
Stress	`test/stress/`	No	Production-like	Performance baselines
Challenge	`challenges/scripts/`	No	Real APIs	Script-based validation
Performance	`test/performance/`	No	Isolated environment	Benchmark baselines

10.3 Challenge Script Specifications

```bash
#!/bin/bash
# challenges/scripts/helixtranslate_verifier_integration.sh
#
# Test: LLMsVerifier integration into HelixTranslate
# Validates: Real provider discovery, verification, scoring

# Color codes
GREEN='\033[0;32m'
RED='\033[0;31m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m'

TESTS_PASSED=0
TESTS_FAILED=0

print_test() { echo -e "${YELLOW}TEST${NC} $1"; }
pass_test() { echo -e "${GREEN}PASS${NC} $1"; TESTS_PASSED=$
        ((TESTS_PASSED+1)); }
fail_test() { echo -e "${RED}FAIL${NC} $1"; TESTS_FAILED=$
        ((TESTS_FAILED+1)); }

# TEST 1: LLMsVerifier module imports successfully
```

```
print_test "Module import"
go run cmd/api-server/main.go --help > /dev/null 2>&1
if [ $? -eq 0 ]; then pass_test "Module imports"; else fail_test
        "Module imports"; fi

# TEST 2: Config loads with verifier section
print_test "Config loading with verifier section"
./bin/api-server -config configs/verifier.yaml &
SERVER_PID=$!
sleep 2
curl -s http://localhost:8080/api/v1/verifier/health | grep -q
        "healthy"
if [ $? -eq 0 ]; then pass_test "Config loads"; else fail_test
        "Config loads"; fi
kill $SERVER_PID

# TEST 3: Provider discovery returns real models
print_test "Provider discovery with live API"
curl -s http://localhost:8080/api/v1/verifier/providers | jq
        '.providers | length' > /dev/null
COUNT=$(curl -s http://localhost:8080/api/v1/verifier/providers |
        jq '.providers | length')
if [ "$COUNT" -gt 0 ]; then pass_test "Provider discovery ($COUNT
        providers)"; else fail_test "Provider discovery"; fi

# TEST 4: Verification pipeline runs
print_test "Verification pipeline"
MODEL_ID="gpt-4"
RESULT=$(curl -s -X POST http://localhost:8080/api/v1/verifier/
        models/$MODEL_ID/verify)
echo "$RESULT" | grep -q "verified"
if [ $? -eq 0 ]; then pass_test "Verification pipeline"; else
        fail_test "Verification pipeline"; fi

# TEST 5: Score adapter returns normalized scores
print_test "Score normalization"
SCORE=$(curl -s http://localhost:8080/api/v1/verifier/models/
        $MODEL_ID | jq '.score')
if (( $(echo "$SCORE > 0" | bc -l) )); then pass_test "Score
        normalized ($SCORE)"; else fail_test "Score normalized";
        fi

# TEST 6: Translation uses verified provider
print_test "Translation with verified provider"
./bin/unified-translator -input test/fixtures/sample.fb2
        -provider openai -verifier-enabled | grep -q "Translation
        completed"
if [ $? -eq 0 ]; then pass_test "Verified translation"; else
        fail_test "Verified translation"; fi

# Summary
echo ""
echo "================================"
echo "Tests passed: $TESTS_PASSED"
echo "Tests failed: $TESTS_FAILED"
```

```
echo "================================"

if [ $TESTS_FAILED -eq 0 ]; then exit 0; else exit 1; fi
```

10.4 Unit Test Requirements (Per Component)

```
// internal/verifier/startup_test.go
func TestStartupVerifier_VerifyAllProviders(t *testing.T) {
    // Setup: Create verifier with test config
    // Execute: Run verification
    // Assert: Results contain verified providers with scores > 0
    // Assert: No mock clients used
}

func TestStartupVerifier_SelectDebateTeam(t *testing.T) {
    // Setup: Pre-populate with scored providers
    // Execute: Select team
    // Assert: Team size <= 25
    // Assert: All selected providers have score >= min threshold
}

// internal/services/llmsverifier_score_adapter_test.go
func TestScoreAdapter_GetProviderScore(t *testing.T) {
    // Setup: Initialize adapter with known scores
    // Execute: GetProviderScore("openai")
    // Assert: Returns normalized value between 0-10
    // Assert: Consistent across calls (cache hit)
}

func TestScoreAdapter_RefreshScores(t *testing.T) {
    // Setup: Initialize adapter
    // Execute: RefreshScores()
    // Assert: lastRefresh updated
    // Assert: scores populated
}

// pkg/translator/llm/llmsverifier_test.go
func TestLLMsVerifierClient_Translate(t *testing.T) {
    // Setup: Create client with test config
    // Execute: Translate(ctx, "Hello", "Translate to French")
    // Assert: Returns non-empty string
    // Assert: No errors
}
```

10.5 Anti-Puff Checklist

For each integration point, the following MUST be true:

- [ ] **Real API calls**: Unit tests may mock; ALL other test types use live provider APIs

- [ ] **Real database**: SQLite file created and queried; WAL mode verified
- [ ] **Real scores**: Scores come from actual LLMsVerifier calculations, not hardcoded
- [ ] **Real provider discovery**: Discovered models match actual provider API responses
- [ ] **Code visibility test**: "Do you see my code?" test runs against real models
- [ ] **No hardcoded fallbacks**: All model lists come from discovery, not static config
- [ ] **Coverage ratchet**: Each PR increases or maintains coverage; never decreases
- [ ] **Challenge validation**: Every feature has a bash challenge script that runs against the live system
- [ ] **Terminal output**: Definition of Done requires pasted terminal output from real runs
- [ ] **No dead code**: All functions called; all imports used; all configs referenced

---

Appendix A: File Inventory

New Files to Create (30+ files)

```
pkg/translator/llm/llmsverifier.go
pkg/translator/llm/groq.go
pkg/translator/llm/cohere.go
pkg/translator/llm/mistral.go
pkg/translator/llm/xai.go
pkg/translator/llm/replicate.go
pkg/translator/llm/cerebras.go
pkg/translator/llm/cloudflare.go
pkg/translator/llm/siliconflow.go
pkg/translator/llm/hyperbolic.go
pkg/translator/llm/togetherai.go
pkg/translator/llm/sambanova.go
pkg/translator/llm/kimi.go
pkg/translator/llm/novita.go
pkg/translator/llm/nlpcloud.go
pkg/translator/llm/upstage.go
pkg/translator/llm/sarvam.go
pkg/translator/llm/modal.go
pkg/translator/llm/publicai.go
pkg/translator/llm/nia.go
pkg/translator/llm/vulavula.go
pkg/translator/llm/kimicode.go
```

```
internal/verifier/startup.go
internal/verifier/config.go
internal/verifier/discovery.go
internal/verifier/scoring.go
internal/verifier/enhanced_scoring.go
internal/verifier/provider_types.go
internal/verifier/health.go
internal/verifier/verification.go
internal/verifier/adapters/provider_adapter.go
internal/services/llmsverifier_score_adapter.go
pkg/verification/llmsverifier_backend.go
pkg/mcp/client.go
pkg/lsp/client.go
pkg/acp/client.go
pkg/embeddings/client.go
pkg/rag/engine.go
pkg/skills/registry.go
pkg/plugin/registry.go
configs/verifier.yaml
```

Files to Modify (13 files)

```
go.mod
pkg/translator/llm/llm.go
internal/config/config.go
pkg/api/handler.go
pkg/grpc/translator.proto
cmd/api-server/main.go
cmd/unified-translator/main.go
pkg/verification/verifier.go
config.json
AGENTS.md
CLAUDE.md
Makefile
.gitmodules
```

Files to Delete (0 files)

No files need deletion.

Appendix B: Module Dependencies

```
digital.vasic.translator (HelixTranslate)
  +-- digital.vasic.llmsverifier (LLMsVerifier)
  |     +-- github.com/gin-gonic/gin v1.11.0
  |     +-- github.com/spf13/cobra v1.10.2
  |     +-- github.com/spf13/viper v1.21.0
  |     +-- github.com/mattn/go-sqlite3 v1.14.32
  |     +-- github.com/golang-jwt/jwt/v5 v5.3.0
  |     +-- github.com/gorilla/websocket v1.5.3
```

```
|     +-- github.com/andybalholm/brotli v1.2.0
|     +-- golang.org/x/crypto v0.46.0
|     +-- google.golang.org/grpc v1.76.0
|     +-- ...
+-- github.com/gin-gonic/gin v1.11.0 (shared)
+-- github.com/golang-jwt/jwt/v5 v5.3.0 (shared)
+-- github.com/google/uuid v1.6.0 (shared)
+-- github.com/gorilla/websocket v1.5.3 (shared)
+-- github.com/lib/pq v1.10.9
+-- github.com/redis/go-redis/v9 v9.7.0
+-- github.com/quic-go/quic-go v0.56.0
+-- google.golang.org/grpc v1.77.0 (version mismatch!)
```

Dependency Conflicts:

- `google.golang.org/grpc`: HelixTranslate uses v1.77.0, LLMsVerifier uses v1.76.0
  - Resolution: Upgrade LLMsVerifier or downgrade HelixTranslate to shared version
- `github.com/gin-gonic/gin`: Both use v1.11.0 (compatible)
- `github.com/gorilla/websocket`: Both use v1.5.3 (compatible)

End of Comprehensive Technical Synthesis Generated from deep repository analysis of HelixTranslate, LLMsVerifier, and HelixAgent